

## 1. Project Title

# Project Overheat: Predictive Maintenance AI

## Data Science Internship Capstone Project

### Objective

To predict catastrophic machine failure 24 hours in advance using IoT sensor telemetry.

### Technologies Used

- Python
- Pandas
- NumPy
- Matplotlib
- Scikit-learn
- Random Forest Classifier

## 2. Import Libraries

In [117...

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score
)

plt.rcParams["figure.figsize"] = (11, 6)

print("Libraries imported successfully.")
```

Libraries imported successfully.

### 3. Load Dataset

```
In [119... data_path = "C:/Users/intel/OneDrive/Desktop/Istudio Data Science/Final Project/iot
df = pd.read_csv(data_path)

print("Dataset loaded successfully.")
print("Shape:", df.shape)

display(df.head())
```

Dataset loaded successfully.

Shape: (10000, 5)

|   | Timestamp              | Sensor_Temp_C | Sensor_Vibration_mm | Sensor_Voltage_V | Catastrophic_Failure |
|---|------------------------|---------------|---------------------|------------------|----------------------|
| 0 | 2023-01-01<br>00:00:00 | 66.490142     | 1.064301            | 220.696572       | 0.0                  |
| 1 | 2023-01-01<br>01:00:00 | 64.585207     | 1.138900            | 220.566647       | 0.0                  |
| 2 | 2023-01-01<br>02:00:00 | 66.943066     | 1.080524            | 218.126960       | 0.0                  |
| 3 | 2023-01-01<br>03:00:00 | 69.569090     | 1.222084            | 221.159168       | 0.0                  |
| 4 | 2023-01-01<br>04:00:00 | 64.297540     | 1.439436            | 217.019835       | 0.0                  |

### 4. Understand Dataset

```
In [122... print("Dataset Information:")
print()

df.info()

print("\nDataset Shape:", df.shape)

print("\nColumn Names:")
print(df.columns.tolist())
```

Dataset Information:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 5 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Timestamp                             10000 non-null  object
1   Sensor_Temp_C                         10000 non-null  float64
2   Sensor_Vibration_mm                   10000 non-null  float64
3   Sensor_Voltage_V                      10000 non-null  float64
4   Catastrophic_Failure                  10000 non-null  float64
dtypes: float64(4), object(1)
memory usage: 390.8+ KB

Dataset Shape: (10000, 5)

Column Names:
['Timestamp', 'Sensor_Temp_C', 'Sensor_Vibration_mm', 'Sensor_Voltage_V', 'Catastrophic_Failure']
```

# 5. Check Missing Values

In [125...

```
missing_values = df.isnull().sum()

print("Missing Values:")
display(missing_values.to_frame("Missing Count"))

print("\nTotal Missing Values:", df.isnull().sum().sum())
```

Missing Values:

|                      | Missing Count |
|----------------------|---------------|
| Timestamp            | 0             |
| Sensor_Temp_C        | 0             |
| Sensor_Vibration_mm  | 0             |
| Sensor_Voltage_V     | 0             |
| Catastrophic_Failure | 0             |

Total Missing Values: 0

# 6. Convert Timestamp

In [128...

```
df["Timestamp"] = pd.to_datetime(
    df["Timestamp"],
    errors="raise"
)

df = df.sort_values(
```

```
    "Timestamp"
).reset_index(drop=True)

print("Timestamp converted successfully.")
print("First timestamp:", df["Timestamp"].min())
print("Last timestamp:", df["Timestamp"].max())
```

Timestamp converted successfully.  
First timestamp: 2023-01-01 00:00:00  
Last timestamp: 2024-02-21 15:00:00

# 7. Check Failure Distribution

```
In [131... print("Original Failure Distribution:")

failure_counts = (
    df["Catastrophic_Failure"]
    .value_counts()
    .sort_index()
)

display(
    failure_counts.to_frame("Count")
)

print(
    "\nFailure Rate:",
    round(
        df["Catastrophic_Failure"].mean() * 100,
        3
    ),
    "%"
)
```

Original Failure Distribution:

| Count                |      |
|----------------------|------|
| Catastrophic_Failure |      |
| 0.0                  | 9985 |
| 1.0                  | 15   |

Failure Rate: 0.15 %

# 8. Phase 1: Temperature Rolling Mean

```
In [134... df["Temp_Rolling_Mean_12h"] = (
    df["Sensor_Temp_C"]
    .rolling(window=12)
    .mean()
)
```

```
display(
  df[
    [
      "Timestamp",
      "Sensor_Temp_C",
      "Temp_Rolling_Mean_12h"
    ]
  ].head(15)
)
```

|    | Timestamp           | Sensor_Temp_C | Temp_Rolling_Mean_12h |
|----|---------------------|---------------|-----------------------|
| 0  | 2023-01-01 00:00:00 | 66.490142     | NaN                   |
| 1  | 2023-01-01 01:00:00 | 64.585207     | NaN                   |
| 2  | 2023-01-01 02:00:00 | 66.943066     | NaN                   |
| 3  | 2023-01-01 03:00:00 | 69.569090     | NaN                   |
| 4  | 2023-01-01 04:00:00 | 64.297540     | NaN                   |
| 5  | 2023-01-01 05:00:00 | 64.297589     | NaN                   |
| 6  | 2023-01-01 06:00:00 | 69.737638     | NaN                   |
| 7  | 2023-01-01 07:00:00 | 67.302304     | NaN                   |
| 8  | 2023-01-01 08:00:00 | 63.591577     | NaN                   |
| 9  | 2023-01-01 09:00:00 | 66.627680     | NaN                   |
| 10 | 2023-01-01 10:00:00 | 63.609747     | NaN                   |
| 11 | 2023-01-01 11:00:00 | 63.602811     | 65.887866             |
| 12 | 2023-01-01 12:00:00 | 65.725887     | 65.824178             |
| 13 | 2023-01-01 13:00:00 | 59.260159     | 65.380424             |
| 14 | 2023-01-01 14:00:00 | 59.825247     | 64.787272             |

## 9. Phase 1: Vibration Rolling Standard Deviation

In [137...

```
df["Vibration_Rolling_Std_12h"] = (
  df["Sensor_Vibration_mm"]
    .rolling(window=12)
    .std()
)

display(
  df[
    [
      "Timestamp",
      "Sensor_Vibration_mm",
```

```

        "Vibration_Rolling_Std_12h"
    ]
].head(15)
)

```

|    | Timestamp           | Sensor_Vibration_mm | Vibration_Rolling_Std_12h |
|----|---------------------|---------------------|---------------------------|
| 0  | 2023-01-01 00:00:00 | 1.064301            | NaN                       |
| 1  | 2023-01-01 01:00:00 | 1.138900            | NaN                       |
| 2  | 2023-01-01 02:00:00 | 1.080524            | NaN                       |
| 3  | 2023-01-01 03:00:00 | 1.222084            | NaN                       |
| 4  | 2023-01-01 04:00:00 | 1.439436            | NaN                       |
| 5  | 2023-01-01 05:00:00 | 1.045792            | NaN                       |
| 6  | 2023-01-01 06:00:00 | 1.400164            | NaN                       |
| 7  | 2023-01-01 07:00:00 | 1.043666            | NaN                       |
| 8  | 2023-01-01 08:00:00 | 1.030475            | NaN                       |
| 9  | 2023-01-01 09:00:00 | 1.363719            | NaN                       |
| 10 | 2023-01-01 10:00:00 | 1.384387            | NaN                       |
| 11 | 2023-01-01 11:00:00 | 1.370282            | 0.164662                  |
| 12 | 2023-01-01 12:00:00 | 0.936841            | 0.178793                  |
| 13 | 2023-01-01 13:00:00 | 1.106810            | 0.180102                  |
| 14 | 2023-01-01 14:00:00 | 1.364598            | 0.181347                  |

## 10. Remove Rolling NaN Values

In [140]...

```

before = len(df)

df = df.dropna(
    subset=[
        "Temp_Rolling_Mean_12h",
        "Vibration_Rolling_Std_12h"
    ]
).reset_index(drop=True)

after = len(df)

print("Rows before removing rolling NaN:", before)
print("Rows after removing rolling NaN:", after)
print("Rows removed:", before - after)

```

Rows before removing rolling NaN: 10000

Rows after removing rolling NaN: 9989

Rows removed: 11

## 11. Visualize Temperature

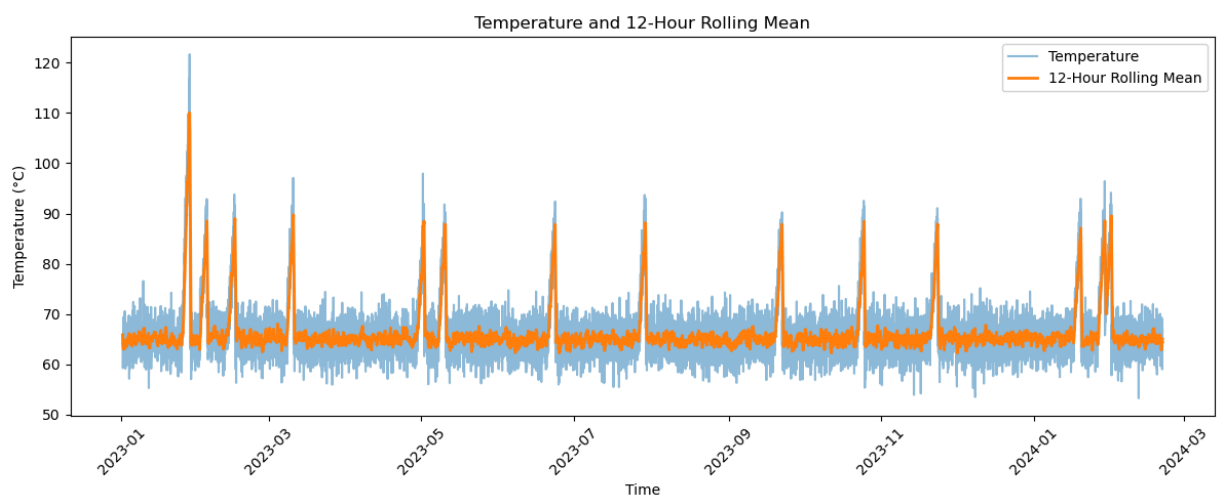
```
In [143... plt.figure(figsize=(12, 5))

plt.plot(
    df["Timestamp"],
    df["Sensor_Temp_C"],
    label="Temperature",
    alpha=0.5
)

plt.plot(
    df["Timestamp"],
    df["Temp_Rolling_Mean_12h"],
    label="12-Hour Rolling Mean",
    linewidth=2
)

plt.title("Temperature and 12-Hour Rolling Mean")
plt.xlabel("Time")
plt.ylabel("Temperature (°C)")
plt.legend()

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



## 12. Visualize Vibration

```
In [146... plt.figure(figsize=(12, 5))

plt.plot(
```

```

df["Timestamp"],
df["Sensor_Vibration_mm"],
label="Vibration",
alpha=0.5
)

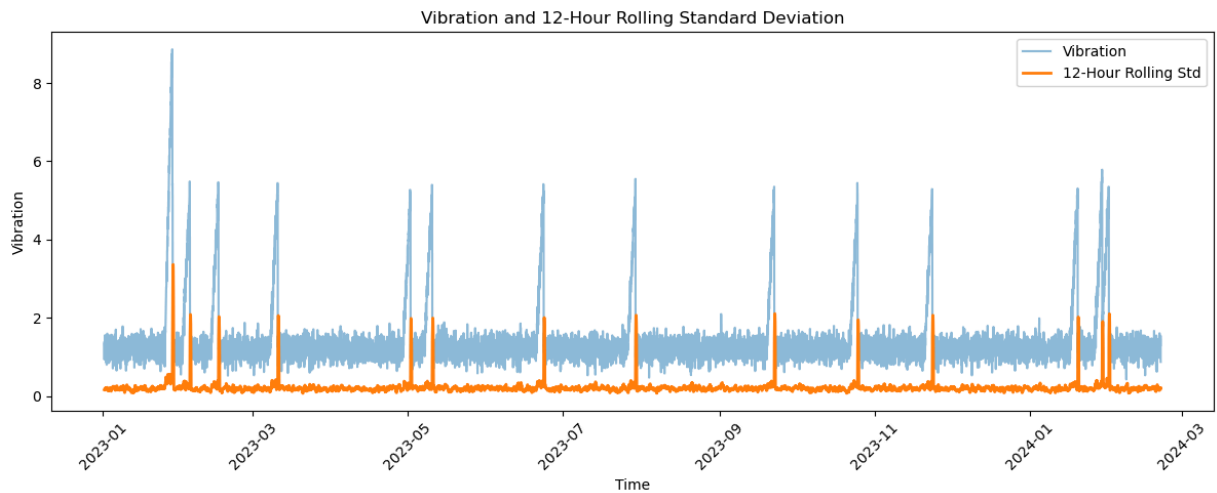
plt.plot(
df["Timestamp"],
df["Vibration_Rolling_Std_12h"],
label="12-Hour Rolling Std",
linewidth=2
)

plt.title(
    "Vibration and 12-Hour Rolling Standard Deviation"
)

plt.xlabel("Time")
plt.ylabel("Vibration")
plt.legend()

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```



## 13. Phase 2: Create 24-Hour Target

In [149...

```

df["Failure_In_24_Hours"] = (
    df["Catastrophic_Failure"]
    .shift(-24)
)

display(
    df[
        "Timestamp",
        "Catastrophic_Failure",
        "Failure_In_24_Hours"
    ]
)

```



```
]
].head(10)
)
```

|   | Timestamp           | Catastrophic_Failure | Failure_In_24_Hours |
|---|---------------------|----------------------|---------------------|
| 0 | 2023-01-01 11:00:00 | 0.0                  | 0.0                 |
| 1 | 2023-01-01 12:00:00 | 0.0                  | 0.0                 |
| 2 | 2023-01-01 13:00:00 | 0.0                  | 0.0                 |
| 3 | 2023-01-01 14:00:00 | 0.0                  | 0.0                 |
| 4 | 2023-01-01 15:00:00 | 0.0                  | 0.0                 |
| 5 | 2023-01-01 16:00:00 | 0.0                  | 0.0                 |
| 6 | 2023-01-01 17:00:00 | 0.0                  | 0.0                 |
| 7 | 2023-01-01 18:00:00 | 0.0                  | 0.0                 |
| 8 | 2023-01-01 19:00:00 | 0.0                  | 0.0                 |
| 9 | 2023-01-01 20:00:00 | 0.0                  | 0.0                 |

## 14. Remove Final 24 Rows

In [152...

```
print(
    "Rows before target cleaning:",
    len(df)
)

df = df.dropna(
    subset=["Failure_In_24_Hours"]
).reset_index(drop=True)

df["Failure_In_24_Hours"] = (
    df["Failure_In_24_Hours"]
    .astype(int)
)

print(
    "Rows after target cleaning:",
    len(df)
)
```

Rows before target cleaning: 9989

Rows after target cleaning: 9965

## 15. Prevent Target Leakage

In [155...

```
df_model = df.drop(
    columns=[
```

```

        "Catastrophic_Failure"
    ]
)

print("Original failure column removed.")
print("\nRemaining columns:")

print(df_model.columns.tolist())

```

Original failure column removed.

Remaining columns:

```
['Timestamp', 'Sensor_Temp_C', 'Sensor_Vibration_mm', 'Sensor_Voltage_V', 'Temp_Rolling_Mean_12h', 'Vibration_Rolling_Std_12h', 'Failure_In_24_Hours']
```

## 16. Define Features and Target

```

In [158... feature_columns = [
    "Sensor_Temp_C",
    "Sensor_Vibration_mm",
    "Sensor_Voltage_V",
    "Temp_Rolling_Mean_12h",
    "Vibration_Rolling_Std_12h"
]

X = df_model[feature_columns]

y = df_model[
    "Failure_In_24_Hours"
]

print("Features:")
print(X.columns.tolist())

print("\nTarget:")
print(y.name)

print("\nX shape:", X.shape)
print("y shape:", y.shape)

```

Features:

```
['Sensor_Temp_C', 'Sensor_Vibration_mm', 'Sensor_Voltage_V', 'Temp_Rolling_Mean_12h', 'Vibration_Rolling_Std_12h']
```

Target:

Failure\_In\_24\_Hours

X shape: (9965, 5)

y shape: (9965,)

## 17. Target Distribution

```
In [161... target_distribution = (
    y.value_counts()
    .sort_index()
)

display(
    target_distribution.to_frame(
        "Count"
    )
)

print(
    "\nFailure percentage:",
    round(y.mean() * 100, 3),
    "%"
)
```

|                     | Count |
|---------------------|-------|
| Failure_In_24_Hours |       |
| 0                   | 9950  |
| 1                   | 15    |

Failure percentage: 0.151 %

## 18. Phase 3: Chronological 80/20 Split

```
In [164... split_index = int(
    len(df_model) * 0.80
)

X_train = X.iloc[
    :split_index
]

X_test = X.iloc[
    split_index:
]

y_train = y.iloc[
    :split_index
]

y_test = y.iloc[
    split_index:
]

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

print(
    "\nTraining failures:",
```

```
y_train.sum()
)

print(
    "Testing failures:",
    y_test.sum()
)
```

Training samples: 7972

Testing samples: 1993

Training failures: 12

Testing failures: 3

## 19. Verify Chronological Split

In [167...

```
print(
    "Training period:"
)

print(
    df.iloc[:split_index]["Timestamp"].min()
)

print(
    "to"
)

print(
    df.iloc[:split_index]["Timestamp"].max()
)

print(
    "\nTesting period:"
)

print(
    df.iloc[split_index:]["Timestamp"].min()
)

print(
    "to"
)

print(
    df.iloc[split_index:]["Timestamp"].max()
)
```

Training period:  
2023-01-01 11:00:00  
to  
2023-11-29 14:00:00

Testing period:  
2023-11-29 15:00:00  
to  
2024-02-20 15:00:00

## 20. Phase 4: Random Forest

```
In [170... model = RandomForestClassifier(  
    n_estimators=300,  
    class_weight="balanced",  
    random_state=42,  
    n_jobs=-1,  
    min_samples_leaf=8,  
    max_features="sqrt"  
)  
  
model.fit(  
    X_train,  
    y_train  
)  
  
print(  
    "Random Forest model trained successfully."  
)
```

Random Forest model trained successfully.

## 21. Predictions

```
In [173... y_pred = model.predict(  
    X_test  
)  
  
y_probability = (  
    model.predict_proba(X_test)[: , 1]  
)  
  
print("Predictions generated successfully.")  
  
display(  
    pd.DataFrame({  
        "Actual": y_test.values,  
        "Predicted": y_pred,  
        "Failure_Probability": y_probability  
    }).head(20)  
)
```

Predictions generated successfully.

|    | Actual | Predicted | Failure_Probability |
|----|--------|-----------|---------------------|
| 0  | 0      | 0         | 0.0                 |
| 1  | 0      | 0         | 0.0                 |
| 2  | 0      | 0         | 0.0                 |
| 3  | 0      | 0         | 0.0                 |
| 4  | 0      | 0         | 0.0                 |
| 5  | 0      | 0         | 0.0                 |
| 6  | 0      | 0         | 0.0                 |
| 7  | 0      | 0         | 0.0                 |
| 8  | 0      | 0         | 0.0                 |
| 9  | 0      | 0         | 0.0                 |
| 10 | 0      | 0         | 0.0                 |
| 11 | 0      | 0         | 0.0                 |
| 12 | 0      | 0         | 0.0                 |
| 13 | 0      | 0         | 0.0                 |
| 14 | 0      | 0         | 0.0                 |
| 15 | 0      | 0         | 0.0                 |
| 16 | 0      | 0         | 0.0                 |
| 17 | 0      | 0         | 0.0                 |
| 18 | 0      | 0         | 0.0                 |
| 19 | 0      | 0         | 0.0                 |

## 22. Classification Report

In [176...

```
print(
    classification_report(
        y_test,
        y_pred,
        zero_division=0
    )
)
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 1.00      | 1.00   | 1.00     | 1990    |
| 1            | 0.33      | 0.67   | 0.44     | 3       |
| accuracy     |           |        | 1.00     | 1993    |
| macro avg    | 0.67      | 0.83   | 0.72     | 1993    |
| weighted avg | 1.00      | 1.00   | 1.00     | 1993    |

## 23. Metrics

```
In [179... accuracy = (
    y_pred == y_test
).mean()

precision = precision_score(
    y_test,
    y_pred,
    zero_division=0
)

recall = recall_score(
    y_test,
    y_pred,
    zero_division=0
)

f1 = f1_score(
    y_test,
    y_pred,
    zero_division=0
)

print(f"Accuracy : {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall   : {recall:.4f}")
print(f"F1 Score : {f1:.4f}")

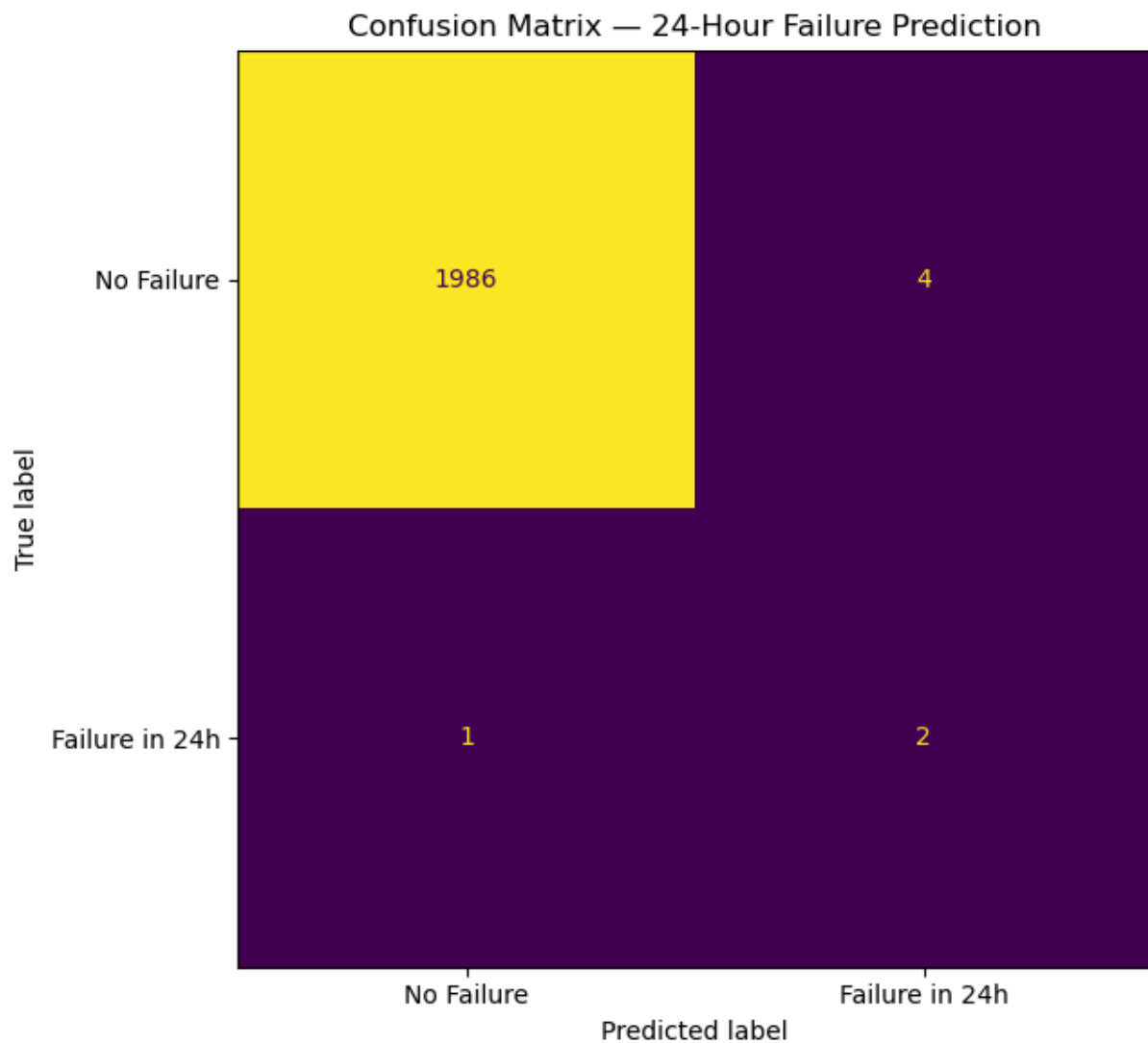
if y_test.nunique() == 2:
    roc_auc = roc_auc_score(
        y_test,
        y_probability
    )
    print(f"ROC-AUC : {roc_auc:.4f}")
```

```
Accuracy : 0.9975
Precision: 0.3333
Recall   : 0.6667
F1 Score : 0.4444
ROC-AUC  : 0.9963
```

## 24. Confusion Matrix

```
In [182... cm = confusion_matrix(  
    y_test,  
    y_pred  
)  
  
fig, ax = plt.subplots()  
  
ConfusionMatrixDisplay(  
    confusion_matrix=cm,  
    display_labels=[  
        "No Failure",  
        "Failure in 24h"  
    ]  
) .plot(  
    ax=ax,  
    values_format="d",  
    colorbar=False  
)  
  
ax.set_title(  
    "Confusion Matrix – 24-Hour Failure Prediction"  
)  
  
plt.tight_layout()  
plt.show()
```





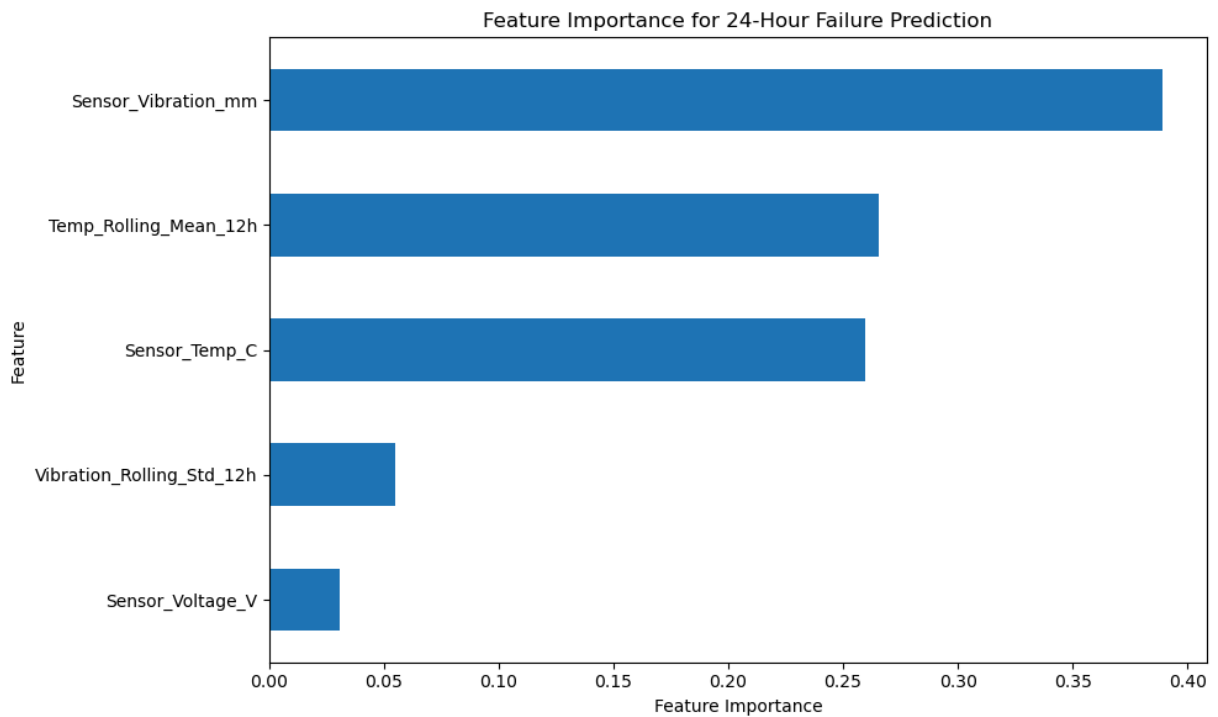
## 25. Feature Importance

```
In [185... feature_importance = pd.Series(  
    model.feature_importances_,  
    index=X_train.columns  
)  
.sort_values(  
    ascending=False  
)  
  
display(  
    feature_importance.to_frame(  
        "Importance"  
    )  
)
```

|                           | Importance |
|---------------------------|------------|
| Sensor_Vibration_mm       | 0.388978   |
| Temp_Rolling_Mean_12h     | 0.265527   |
| Sensor_Temp_C             | 0.259787   |
| Vibration_Rolling_Std_12h | 0.055106   |
| Sensor_Voltage_V          | 0.030603   |

## 26. Feature Importance Chart

```
In [188... importance_sorted = (  
    feature_importance  
    .sort_values()  
)  
  
plt.figure(  
    figsize=(10, 6)  
)  
  
importance_sorted.plot(  
    kind="barh"  
)  
  
plt.title(  
    "Feature Importance for 24-Hour Failure Prediction"  
)  
  
plt.xlabel(  
    "Feature Importance"  
)  
  
plt.ylabel(  
    "Feature"  
)  
  
plt.tight_layout()  
plt.show()
```



## 27. Business Recommendation

```
In [191... ranked_features = (
    feature_importance
    .sort_values(
        ascending=False
    )
)

top_feature = ranked_features.index[0]
second_feature = ranked_features.index[1]
third_feature = ranked_features.index[2]

print("=" * 70)
print("FINAL BUSINESS RECOMMENDATION")
print("=" * 70)

print(
    f"\n1. Primary indicator to monitor: {top_feature}"
)

print(
    f"2. Secondary indicator to monitor: {second_feature}"
)

print(
    f"3. Supporting indicator: {third_feature}"
)

print(
    "\nMaintenance Recommendation:"
```

```

)

print(
    "Machines receiving a high-risk prediction should be inspected "
    "before the predicted 24-hour failure window."
)

print(
    "This allows maintenance teams to plan inspection or maintenance "
    "before a catastrophic motor failure occurs."
)

print(
    "\nThe model should be monitored continuously using failure-class "
    "recall and precision rather than accuracy alone."
)

```

```

=====
FINAL BUSINESS RECOMMENDATION
=====

```

1. Primary indicator to monitor: Sensor\_Vibration\_mm
2. Secondary indicator to monitor: Temp\_Rolling\_Mean\_12h
3. Supporting indicator: Sensor\_Temp\_C

Maintenance Recommendation:

Machines receiving a high-risk prediction should be inspected before the predicted 24-hour failure window.

This allows maintenance teams to plan inspection or maintenance before a catastrophic motor failure occurs.

The model should be monitored continuously using failure-class recall and precision rather than accuracy alone.

## 28. Final Summary

### Project Outcome

The Predictive Maintenance AI pipeline successfully:

1. Loaded and cleaned IoT telemetry data.
2. Processed the timestamp chronologically.
3. Created 12-hour rolling temperature and vibration features.
4. Shifted the failure target 24 hours into the future.
5. Prevented target leakage.
6. Used an 80/20 chronological train-test split.
7. Trained a balanced Random Forest Classifier.
8. Evaluated the model using classification metrics.
9. Visualized the confusion matrix.
10. Extracted Random Forest feature importances.

11. Generated a maintenance recommendation.

## Business Impact

The proposed system can help maintenance teams move from:

**Reactive Maintenance → Predictive Maintenance**

Instead of waiting for a motor to fail, the organization can identify high-risk machines in advance and investigate them before catastrophic failure.

**--Project Completed--**

In [ ]:

In [ ]:

In [ ]:

In [ ]: